Visualizador de Backpropagation & Otimizadores

Projeto de Cálculo 3 | Ciência da Computação
Relatório de progresso, versão 2 (Etapas 1 a 13 e correções)

1. Estado atual

Ambiente de jogo 2D em JavaScript puro (ES6+), sem bibliotecas de IA. A rede neural aprende por **imitation learning**: o humano grava pares [estado, ação], duas redes com os mesmos pesos iniciais (SGD e Adam) treinam em paralelo nos mesmos mini-batches, e a interface mostra o diagrama da rede, a Jacobiana, as equações com valores reais e a curva de Loss.

O código planejado está completo (13 arquivos). O app roda via `python3 -m http.server 8000`. A IA já apresenta aprendizado (Loss bem abaixo do chute aleatório), e o que resta é validação, entrega e alinhamento com o grupo (seção 5).

2. Estrutura de arquivos

Arquivo	Responsabilidade
index.html / styles/main.css	Grid 2x2, botões: Gravar, Treinar, Otimizador, Ativação, Painel, Verificar Gradiente, Exportar/Importar dataset, Resetar
src/main.js	Integração: fases, 2 redes em paralelo, treino por mini-batch (gradiente médio), painéis
src/math/matrix.js, activations.js	Álgebra linear; sigmoid, ReLU, softmax e Jacobiana do softmax
src/math/nn.js	forward/backward (regra da cadeia), clone(), ativação oculta sigmoid ou ReLU
src/math/optimizers.js	SGD e Adam
src/math/gradcheck.js	Verificação numérica do gradiente (diferença central)
src/game/player.js, engine.js	Física, colisão AABB, limites horizontais; estado de 4 entradas
src/game/telemetry.js	Gravação one-hot, contagem por classe, amostragem balanceada, import validado
src/ui/visualizer.js, charts.js	Diagrama com onda reversa; Loss suavizada, janela ~50 s e linha do chute aleatório

3. O que foi feito

Etapas 1 a 9 (núcleo matemático, jogo, telemetria, visualização e integração) estão descritas no relatório anterior. Abaixo, o que veio depois.

Etapa	O que entrou	Big-O / observação
10	Gradiente **médio** (antes somado: passo efetivo era lr × 8); backward devolve as ativações (sem forward duplicado); gravação de "none" 1 a cada 8 frames; painel da Jacobiana 4×4 com rótulos e normas do gradiente	trainStep: $O(B \cdot \text{soma } n_l \cdot n_{l+1})$; softmaxJacobian: $O(n^2)$
11	Gradient checking; SGD e Adam em paralelo (clone() garante mesmos pesos iniciais e mesmos batches); Loss suavizada com legenda numérica	gradientCheck: $O(P \cdot \text{soma } n_l \cdot n_{l+1})$, só ao clicar
12	Onda reversa do backprop (saída para entrada, brilho proporcional a |δ| com máximo global); ativação ReLU com inicialização He, mantendo o dataset ao trocar	diagrama: O(N + E)
13	Painel de equações vivas (perda, δ, gradiente e passo de atualização do SGD/Adam com números reais); exportar/importar dataset em JSON validado; README com roteiro de apresentação	load: O(n·k), k = 4
Correções	Limite horizontal do jogador; **estado com 4 entradas** (posição x incluída; rede [4, 6, 4]); "jump" gravado só no frame em que o pulo acontece (no chão); contagem por classe no HUD; gráfico com 15 frames por ponto	getState: O(1); forward: 48 multiplicações contra 42

4. Bugs encontrados e corrigidos (acumulado)

Sintoma	Causa	Correção
Números da Jacobiana borrados	Canvas esticado por CSS sem ajustar a resolução	devicePixelRatio e clearRect a cada frame
Jogador invisível	GROUND_Y fixo fora da altura real do canvas	groundY calculado a partir do canvas
Painéis vazios (várias vezes)	engine.js com comentários no lugar dos métodos; const batch duplicado; telemetry.js sem export ACTIONS	Arquivos completos; conferir com grep e com o console (F12)
IA sempre prevendo "none"	Dataset desbalanceado	Amostragem estratificada por classe
Loss muito ruidosa	Gradiente somado em vez de médio	Divisão por batch.length
Jogador sumia pela borda	Nada limitava x	Clamp horizontal em update()
IA presa na parede esquerda; Loss em platô	Estado sem a posição x: situações diferentes tinham o mesmo vetor	x normalizado no estado (4 entradas)
Dataset: 83% "jump", 0 "left"	Segurar Espaço gravava "jump" em todo frame, e jump tinha prioridade sobre as setas	jump só no chão; setas valem no ar
Curva sem tendência	Janela de só ~3 s (200 frames)	15 frames por ponto: janela de ~50 s
Tela com versão antiga	Cache do navegador	Janela anônima, Disable cache, outra porta

Resultados observados

Com dataset de 670 amostras (J24 L208 R239 N199): Loss SGD = 0,985 e Adam = 0,733, contra 1,386 (ln 4) do chute aleatório. Como exp(-Loss) é a probabilidade média dada à ação certa, a rede acerta em média ~37% (SGD) e ~48% (Adam), contra 25% do acaso. A Jacobiana exibida confere com $J_{ij} = s_i(\delta_{ij} - s_j)$, e o passo de atualização mostrado nas equações confere com a aritmética manual. Com apenas 24 pulos no dataset, a rede tende a decorar essa classe.

5. O que ainda falta

Quase tudo é validação e entrega; não há mais funcionalidade obrigatória do PDF do projeto sem implementar.

Validação

• **Resultado do gradient check:** ainda não foi relatado. Reduzir o zoom (Ctrl + menos), clicar em Verificar Gradiente e anotar o erro. Esperado: OK, erro relativo menor que 1e-5. É a prova de que a regra da cadeia está correta.

• **Medir a IA:** maior score e número de pulos corretos por minuto, por 2 minutos com SGD e 2 com Adam. Número concreto para a apresentação.

• **Mais pulos no dataset:** passar de ~60 amostras de "jump" para a rede não decorar a classe.

• **Vídeo de 30 a 60 s** do treino rodando, como plano B se a demonstração ao vivo falhar.

Entrega e grupo

• **Repositório pessoal:** concluído (github.com/A7-J/visualizador-backpropagation). Trocar o PDF antigo pelo desta versão e corrigir o e-mail do git config (faltava o ponto em gmail.com).

• **Repositório do grupo:** subir matrix.js e activations.js (a parte atribuída) em src/math/ e o restante em um branch com Pull Request. Avisar os colegas: nn.js e main.js antigos não funcionam com a versão final.

• **Datas dos commits:** usar sempre a data real. Não usar GIT_AUTHOR_DATE nem GIT_COMMITTER_DATE com datas inventadas: o histórico falso é risco acadêmico para o grupo todo, e a hora do push no GitHub não é editável.

• **Créditos do README:** preencher nomes e divisão de arquivos.

• **Cada integrante** deve conseguir explicar o módulo que levou; os testes de mesa servem de material de estudo.

• **Divisão da apresentação:** definir quem apresenta cada painel (roteiro de 5 minutos no README).

Opcional

• Acurácia por classe no painel, para mostrar onde a rede erra.

• Testes unitários de matrix.js e activations.js (usando os testes de mesa como casos).

• Tooltips explicando cada painel; polímento visual (Mês 4 do plano).

6. Roteiro de teste e demonstração

#	Ação	O que esperar
1	Servidor na raiz, janela anônima, F12 aberto	Console sem erros vermelhos (o 404 do favicon é inofensivo)
2	Resetar, Gravar Ações por 60 a 90 s, tocando Espaço 1 vez por obstáculo e alternando as setas	HUD com as 4 classes acima de ~40 (J, L, R, N)
3	Treinar IA e observar 2 a 3 min	Loss desce abaixo da linha tracejada do chute (1,386); onda rosa percorre a rede da saída para a entrada
4	Painel: alternar Jacobiana e Equações	Linhas da Jacobiana somam 0; passo do SGD confere com a conta manual
5	Otimizador: alternar SGD e Adam; Ativação: alternar sigmoid e ReLU	As duas redes continuam treinando; a ReLU deixa a camada de entrada mais forte no diagrama
6	Verificar Gradiente	Erro relativo ~1e-9 e "OK"